

期中复习参考

一、判断题

请判断下列说法是否正确，正确写“√”，错误写“×”。

1. 如果内核映像仍被装载到 0x80200000，但链接脚本把入口代码链接到了另一处地址，那么系统即使总内存充足，也可能在最早期启动阶段就无法正常运行。
2. 从用户态陷入内核态时，RISC-V 硬件会自动把所有通用寄存器逐个保存到内存中的 TrapContext。
3. 一个系统调用一定会导致当前任务被切换出去。
4. 在 uCore/rCore ch4 之后，不同进程的页表仍然可能把各自的某个虚拟页映射到同一个物理页帧。
5. 在 RR 调度中，把时间片设置得越小越好，因为这样既能提高响应性，也不会引入额外代价。
6. 在动态连续内存分配中，即使不做紧凑，只要两个空闲块在地址上相邻，也仍然有可能把它们合并。
7. 覆盖（overlay）技术与分页机制一样，都能让内存换入换出过程对程序员完全透明。
8. TLB 的主要作用是减少地址转换过程中频繁查页表带来的开销。
9. 如果修改了某进程正在使用的页表项，但处理器仍继续使用旧的 TLB 翻译结果，那么访存行为可能与新页表内容不一致。
10. 在 Sv39 中，叶子页表项不一定只出现在最低一级页表中。
11. 用户栈和内核栈承担的职责不同，因此它们不能简单地视作同一块栈空间。
12. 从应用视角看，进程这一抽象同时提供了相对独立的控制流和相对独立的地址空间。
13. HRRN 相比 SJF，更不容易让长时间等待的作业一直得不到服务。
14. 即使时钟中断发生在内核态，操作系统通常也可在后续合适的调度点触发任务切换。
15. 若系统调用返回前如果不改变 sepc 寄存器值，那么返回用户态后可能会重复执行同一条 ecall。
16. 在 uCore/rCore ch4 的地址空间设计中，内核不能仅凭“用户传来的虚拟地址数值”就直接把它当作当前内核可安全访问的指针使用。
17. SRT 和 RR 调度算法都属于可抢占调度，但它们决定何时换下当前任务的依据并不相同。
18. 并发和并行是相同的概念，都是指多个任务在同一时刻同时执行。
19. 在 Sv39 下，只要 TLB 命中，本次地址转换通常无需再走三级页表访存。
20. 子进程必须比父进程先退出。否则，子进程会因为“失去父进程”而永久无法回收。

21. 用户态程序可直接写 `sstatus` 寄存器的 S 态中断控制位来关闭时钟中断。
22. 改进 Clock 优先选择 (A=0, M=0) 页淘汰，体现了“兼顾局部性与写回开销”的折中思想。
23. 时间片轮转 (RR) 中，当时间片足够大时，其行为会退化为先来先服务 (FCFS)。
24. 在本课程 uCore/rCore ch4 的实现中，跳板页 `trampoline` 需要在用户页表和内核页表中映射为同一虚拟地址并指向同一物理页，否则切换 `satp` 时无法保证取指连续。
25. 进程的状态切换“运行 -> 等待”常由进程发起 I/O/sleep 触发；而进程的状态切换“运行 -> 就绪”可由时钟中断触发的抢占导致。

二、选择题

请从每题四个选项中选出一个最符合题意的答案。

1. 在 rCore/uCore ch4 的双页表 trap 路径中，下列哪个“关键步骤顺序”最符合实验代码？
 - A. `ecall` → `usertrap` (Rust/C) → `uservec` (汇编) → 写 `satp` → 返回
 - B. `ecall` → 硬件直接写 `satp=kernel` → `uservec` → `syscall` → 返回
 - C. `ecall` → 硬件写 `sepc/scause` 并跳 `stvec(uservec)` → 保存上下文到 `TrapContext/TrapFrame` → 写 `satp` 切到内核页表并 `sfence.vma` → 跳转内核 `trap` 处理函数
 - D. `ecall` → 直接执行 `sret` 返回用户态，再由用户库转发到内核
2. 在 rCore/uCore 的双页表实现里，把 `TrapContext/TrapFrame` 映射到用户地址空间固定位置（如 `TRAPFRAME`）的主要原因是
 - A. `trap` 入口汇编在切换到内核页表前，必须先有一块“当前可访问”的位置保存寄存器
 - B. 这样用户程序就能直接调试和修改内核寄存器
 - C. 这样可以完全省掉 `sscratch` 的使用
 - D. 这样可避免所有 TLB 刷新
3. 在地址空间切换后执行 `sfence.vma` 的最直接目标是
 - A. 刷新 I-Cache，确保新代码可见
 - B. 提升分支预测命中率
 - C. 把用户页表内容同步写回磁盘

- D. 使旧地址空间相关 TLB 项失效，避免用过期映射完成翻译
4. 系统中有父进程 P 和子进程 C 正在运行。此时，父进程 P 调用 `waitpid(C, ...)`；一段时间后，子进程 C 调用 `exit`。从父进程 P 发起调用开始，下列状态变化最合理的是
- A. P：运行→就绪；C：运行→等待
 - B. P：运行→等待；C：运行→退出（僵尸）；随后 P：等待→就绪
 - C. P：运行→退出；C：运行→就绪
 - D. P：创建→运行；C：就绪→等待
5. 在“全部作业已到达、单处理器、非抢占、只关注平均周转时间”的条件下，理论上最优的是
- A. SJF
 - B. FCFS
 - C. RR
 - D. MLFQ
6. 对一组纯 CPU-bound 进程，若 RR 的时间片 q 远大于所有进程总服务时间，其调度行为最接近
- A. SRT
 - B. HRRN
 - C. FCFS
 - D. 优先级抢占调度
7. 在 Sv39 中，若某虚拟页对应叶子 PTE 的标志为 $V=1$, $R=0$, $W=0$, $X=0$ ，用户态执行 `load` 访问该页时最可能发生的是
- A. 访问成功，因为 $V=1$ 说明该页可读
 - B. 访问成功，因为 $X=0$ 仅影响取指
 - C. 触发异常（如 `load page fault`），因为该映射不允许读访问
 - D. 自动把该页升级为可读页后继续执行
8. 在 Sv39 下，从空页表开始映射连续 256MB（从 $VA=0$ 开始，4KB 页），仅计算“页表本身”至少需要多少个物理页帧？
- A. 64

B. 128

C. 129

D. 130

9. 关于 Belady 异常与页面置换算法，下列说法正确的是

A. FIFO 和 LRU 都不会出现 Belady 异常

B. 只要物理页帧增加，任何算法缺页率都严格下降

C. Clock 一定是栈算法，因此不会出现 Belady 异常

D. LRU 具有栈性质，FIFO 可能出现 Belady 异常

10. 在 rCore/uCore ch4 双页表下实现 `sys_write(fd, user_buf, len)` 时，正确做法是

A. 直接把 `user_buf` 当作内核指针解引用拷贝

B. 按当前进程页表将用户缓冲区翻译成内核可访问的字节片段后再写出

C. 临时关闭分页，再线性拷贝用户缓冲区

D. 强制要求用户缓冲区必须位于跳板页 `trampoline`

11. 在“AI 自动生成代码并在线执行”的教学平台里，以下哪组机制最能构成底层安全边界

A. 地址空间隔离 + 特权级机制 + 系统调用/资源配额约束

B. 仅依赖编译器静态检查

C. 仅禁用网络访问

D. 仅依赖语言运行时沙箱

12. 考虑如下伪代码：

```
pid = fork();
if (pid == 0) {
    exec("hello");
    printf("X\n");
} else {
    waitpid(pid, ...);
    printf("Y\n");
}
```

若 `exec` 成功，下面说法正确的是

A. `x` 和 `y` 都必然打印

- B. x 必然打印, y 不一定打印
 - C. 子进程不会执行 `printf("X");`; 父进程在 `waitpid` 返回后打印 y
 - D. `exec` 成功会导致子进程 PID 改变
13. 下列哪一组最贴近课程中强调的、操作系统向应用提供的主要抽象?
- A. 中断向量表、DMA、缓存
 - B. 进程 (process)、文件 (file)、地址空间 (address space)
 - C. 仅 CPU 寄存器
 - D. 仅物理内存帧
14. 虚拟化在内存管理中的典型体现是:
- A. 所有进程共用同一套物理地址编程
 - B. 各进程拥有独立虚拟地址空间, 由 OS 与 MMU 映射到物理内存
 - C. 关闭 MMU 以提高速度
 - D. 虚拟化仅指虚拟机, 与进程无关
15. 与宏内核 (monolithic kernel) 相比, 微内核 (microkernel) 的常见设计倾向是:
- A. 把所有驱动强行塞进内核
 - B. 尽量把服务放到用户态以缩小可信内核代码
 - C. 取消用户态
 - D. 不再提供系统调用
16. 用户态程序主动请求内核服务时, RISC-V 使用的陷入指令是:
- A. `mret`
 - B. `sret`
 - C. `ecall`
 - D. `ebreak`
17. 下列哪一类操作通常不能在用户态 (U-mode) 随意执行?
- A. 普通算术运算
 - B. 访问用户可访问内存的 `load/store`

- C. 写 satp 等与页表相关的 CSR
- D. jal / jalr
18. 与「一次只驻留一道作业、顺序执行」的简单批处理相比，多道程序 (multiprogramming) 更有利于提高 CPU 利用率，其主要原因是：
- A. 缩短每个作业代码长度
- B. 内存中可同时驻留多道程序，某道等待 I/O 时可执行其他道
- C. 取消特权级
- D. 不再需要加载器
19. 在用户页表中映射 trampoline 代码页时，最合适的权限组合是：
- A. V=1, R=1, W=1, X=1, U=1
- B. V=1, R=1, W=0, X=1, U=0
- C. V=1, R=0, W=1, X=0, U=0
- D. V=1, R=1, W=1, X=0, U=0
20. 在可抢占的分时系统中，下面哪个事件必然导致“运行态 -> 就绪态”而不是“运行态 -> 等待态”？
- A. 阻塞式 read 等待键盘输入
- B. 父进程执行 wait 并成功回收子进程
- C. 进程执行 exit
- D. 时间片耗尽触发时钟中断

三、问答题

1.

某同学为了“精简代码”，做了如下修改：

- 删除了内核入口处的汇编初始化代码，试图让固件直接跳到 main/rust_main
- 把清零 .bss 段的逻辑挪到了第一次串口输出之后

结果造成同一份内核有时一开机就异常，有时串口输出乱码，有时只能运行到很早的阶段就死机。

请回答：

1. 在课程实验的启动路径中，固件与操作系统内核分别承担什么角色？为什么不能把固件简单视为“内核的一部分”来忽略？
2. 在进入 C/Rust 编写的主体函数之前，入口汇编阶段通常至少需要完成哪些关键准备工作？为什么这些工作不能想当然地省掉？
3. 为什么把 .bss 段的清零操作挪到第一次串口输出之后，会导致“有时乱码、有时死机”的异常现象？从启动前置条件/系统正确性的角度，解释这为什么属于原则性错误。
4. 为了分析和修复bug，使用rust编程和使用c编程的同学一起讨论，熟练使用面向riscv-64 处理器的一系列分析与调试工具：qemu，gdb，readelf，file，objdump，objcopy，rust-readobj，rust-objdump，rust-objcopy等，终于解决了问题。请按你使用的具体情况简要说明这些工具的功能和用法。

注意：没用到的工具不用说明，如用C编程的同学不用说明rust-readobj等。

2.

某同学在实 rCore/uCore ch4 里实现 sys_write 时，为了图省事，直接把用户传入的 buf 当成普通指针使用，写出了类似下面的逻辑：

```
copy_from(buf, kernel_buf, len)
```

其中 copy_from 的内部本质上是直接对 buf 做解引用或 memcpy，既不检查地址合法性，也不显式做地址翻译或按页拷贝。

他给出的理由是：“用户传进来的本来就是虚拟地址，内核和用户程序都跑在同一台 CPU 上，只要地址数值对得上，内核就能直接读。”

请回答：

1. 这个理由为什么在早期实验阶段可能“看起来没出问题”，但到了 ch4 就不再成立？
2. 如果用户传入空指针、未映射地址，或者传入了一个跨页的缓冲区，上述写法可能导致哪两类后果？
3. 从机制上说，uCore/rCore ch4 之后的实验，如果内核要正确处理用户缓冲区，至少应当考虑哪些检查或处理步骤？
4. 若因为这个问题触发了 trap，scause、sepc、stval 在定位错误时分别能帮助我们看什么？

3.

某单核教学操作系统中有三个进程：

- A：长时间 CPU 密集型任务
- B：类似交互式命令行任务，每次只运行很短一段就等待输入

- c: 后台服务进程，偶尔会因为等待 I/O 完成而阻塞

对于 FCFS、SJF、SRT、HRRN、RR 这几种单处理器调度策略，请回答：

1. 当 B 因等待输入而暂停、c 因等待 I/O 完成而暂停、随后又被唤醒时，它们在“就绪态、运行态、等待态”之间大致会发生哪些状态变化？
2. 在上述几种算法中，哪些属于可抢占调度，哪些属于不可抢占调度？内核一般会在什么事件发生后真正运行调度器？
3. 如果系统特别关心 B 的交互响应时间，但又不希望 A 和 c 长期被饿死，那么哪一类策略更合适？请说明理由。
4. 为什么 RR 的时间片既不能过大，也不能过小？

4.

某应用的执行明显分阶段：

- 阶段 1 反复访问模块 M1 与数据区 D1
- 阶段 2 突然转向模块 M2 与数据区 D2

机器物理内存较小，系统运行时出现了如下现象：

- 缺页异常越来越频繁
- 磁盘 I/O 占用很高
- CPU 利用率反而下降

请回答：

1. 这种现象最可能说明系统进入了什么状态？它和程序的局部性变化有什么关系？
2. 请比较覆盖（overlay）、交换（swapping）和虚拟存储（virtual memory）三者“由谁管理、以什么为单位、对程序员是否透明”上的主要区别。
3. 为什么 FIFO 这类算法有时会出现“物理页帧更多、效果反而更差”的反常现象？为什么 LRU/Clock 这一类思路通常更贴近程序的近期行为？
4. 工作集（working set）或缺页率页面置换（PFF）这类思路，试图从哪个角度缓解上述问题？核心想法分别是什么？

5.

在 uCore/rCore 实验的 ch4（地址空间）版本中，操作系统采用双页表机制。以下是 trap 处理的部分关键代码（以 uCore 为例）：

```
// trap.c
void usertrap() {
    set_kerneltrap();           // (A)
    struct trapframe *tf = curr_proc()->trapframe;
    uint64 cause = r_scause();  // (B)
    if (cause == UserEnvCall) {
```



```

        tf->epc += 4;                // (C)
        syscall();
    }
    // ...
    usertrapret();
}

// trampoline.S (部分)
uservec:
    csrwr a0, sscratch, a0        // 交换 a0 和 sscratch
    sd ra, 40(a0)                 // 保存用户寄存器到 trapframe
    // ... 保存其余寄存器
    ld sp, 8(a0)                  // 加载内核栈指针
    ld t0, 0(a0)                  // 加载内核页表基址
    csrwr satp, t0                 // (D) 切换到内核页表
    sfence.vma zero, zero         // (E) 刷新 TLB
    ld t0, 16(a0)                 // 加载 usertrap 地址
    jr t0                         // (F) 跳转到 trap 处理函数

```

请回答：

1. 当用户态程序执行 `ecall` 指令触发系统调用时，RISC-V 硬件自动完成哪些操作？（至少列出三项）
2. 在上述 trampoline 代码中，(D) 和 (E) 两行的作用是什么？为什么页表切换必须在这段代码中完成，而不是在进入 `usertrap()` 函数之后再切换？
3. 代码 (C) 处将 `tf->epc` 加 4，即把保存的用户程序计数器前移一条指令。为什么要这样做？举一个“不应该前移”的异常场景作为对比。
4. TrapFrame（中断上下文）在 uCore/rCore 中被映射到用户地址空间的固定位置（TRAPFRAME），而不是保存在内核栈上。请说明为什么要这样设计。（提示：考虑 TrapFrame 保存的时机和此时 CPU 正在使用哪个页表。）
5. trampoline 页在用户页表和内核页表中的虚拟地址映射有什么特点？这个特点为什么是必要的？

6.

在 uCore/rCore 的 ch3（多道程序与分时多任务）版本中，内核栈的定义如下：

```

// uCore ch3
char kstack[16][4096]; // 16个内核栈，每个 4096 字节

```

假设某同学在编写内核代码时，在多个函数中嵌套使用了较大的局部数组：

```

void some_function() {
    int big_array[1024]; // 占用 4096 字节
    // ...
}

```

测试运行后发现，某些不相关函数的返回值被莫名篡改，程序行为变得不可预测。

请回答：

1. 分析上述问题的根因。结合内核栈的内存布局（kstack 是一个连续的 char 数组），说明为什么在一个 4096 字节的内核栈上分配 4096 字节的局部数组会导致其他任务的数据被破坏。
2. 为什么编译器（无论是 GCC 对 C 代码，还是 rustc 对 Rust 代码）无法在编译期检测并阻止这类栈溢出问题？
3. 在升级到 rCore/uCore ch4（引入页表和地址空间）之后，可以采取什么机制来检测栈溢出？说明其基本原理，以及涉及的 RISC-V CSR 寄存器。
4. 如果一个用户态程序访问了一个未映射的虚拟地址（例如空指针解引用），请描述从异常产生到操作系统处理的完整流程，按顺序列出涉及的硬件自动操作和软件处理步骤。